Chapter 11 — Spring Security 2: Tokens and OAuth2

Chapter 10 authenticated people with a password and remembered them with a session cookie. That works for one browser talking to one server. Modern systems have more parties: a mobile app, a single-page app, twenty microservices, a partner's backend, and an identity provider such as Keycloak, Okta, Entra ID or Google that owns the user accounts. They need a way to prove identity **without sharing passwords and without shared sessions**. That way is **tokens**, and the standard for handing them out is **OAuth 2.0** with **OpenID Connect** on top.

This chapter explains the ideas from zero and then proves Spring's behaviour with tests. The signing keys are generated locally, so **no external identity provider is needed** to run anything.

What you'll learn

1. Bearer tokens and JWTs: structure, claims, signatures, keys (JWK/JWKS).
2. OAuth 2.0 and OpenID Connect: roles, grant types, access vs refresh vs ID tokens.
3. A **JWT resource server** in Spring Security 7: validation rules (signature, expiry, issuer, audience, `typ`), 401 `invalid_token` vs 403 `insufficient_scope`.
4. Minting tokens with `NimbusJwtEncoder` (new builder in Spring Security 7).
5. Mapping claims to authorities (`SCOPE_`, roles) and testing with the `jwt()` post-processor, including its biggest pitfall.
6. Opaque tokens and introspection, and multi-tenant resource servers.
7. **OAuth2 login** (the authorization-code flow with PKCE) as a sequence diagram, and the parts we can run.
8. Service-to-service calls (client credentials, `RestClient`, HTTP interfaces, token relay), refresh tokens, logout.
9. Where tokens should live in a browser (BFF), key and secret storage, Spring Authorization Server, rate limiting.
10. **Security fundamentals beyond Spring** (§11.18): encoding vs hashing vs encryption, symmetric vs asymmetric keys, signatures and certificates, the TLS 1.3 handshake, SSH keys, TOTP codes (RFC 6238, runnable), passkeys/WebAuthn, and SSO with SAML vs OIDC.

All code lives in `code/` — three Spring Boot **4.1.1** projects plus one plain-JDK project (Spring Security **7.1.1**, Java 25):

Project	Shows
`jwt-resource-server`	local RSA key, `/token` endpoint with `NimbusJwtEncoder`, a JWT-protected API, every validation failure, `jwt()` tests
`resource-server-variants`	opaque tokens with an introspector, multi-tenant JWTs (two issuers, two keys)
`oauth2-login-client`	OAuth2/OIDC login configuration. The redirect to the provider, `state`/`nonce`/PKCE, and `oidcLogin()` tests
`crypto-basics`	plain JDK: Base64 vs SHA-256 vs AES-GCM vs Ed25519 signatures, an X25519 key exchange, and TOTP checked against the RFC 6238 test vectors (§11.18)

Every output block is real `mvn -q -B -ntp verify` output (log lines trimmed as `...`). Timestamps and random values change on each run.

> **Prerequisites.** The filter chain, 401 vs 403, authorities and CSRF: Chapter 10. Browser storage, cookies and XSS: JavaScript handbook Chapter 14. Gateways and BFFs as architecture patterns: API Gateway, Backend for Frontend.

11.1 From sessions to tokens: bearer tokens and JWT

What it is

A **bearer token** is a string that grants access to whoever *bears* (holds) it, like a cinema ticket. The server does not ask who you are, only whether the ticket is valid. It travels in a header:

```
GET /api/messages HTTP/1.1
Authorization: Bearer eyJraWQiOiJkZW1vLWtleS0xIiwidH...
```

A **JWT (JSON Web Token, RFC 7519)** is the most common token format. It is three Base64URL-encoded parts joined by dots, `header.payload.signature`. Here is a token our `/token` endpoint issued, decoded:

```
token (3 base64url parts) : eyJraWQiOiJkZW1vLWtleS0xIiwidH... length=670
header  : {"kid":"demo-key-1","typ":"JWT","alg":"RS256"}
payload : {"sub":"bob","aud":"notes-api","scope":"messages.read messages.write","roles":
["ADMIN"],"iss":"https://auth.demo.test","exp":<epoch>,"iat":<epoch>,"jti":"<uuid>"}
```

Part	Contains	Notes
Header	`alg` (signature algorithm), `kid` (key id), `typ`	tells the receiver *how* to verify
Payload	**claims**: statements about the subject	readable by anyone, since it is only encoded, not encrypted
Signature	signature over `header.payload`	proves who issued it and that nothing changed

The registered claims you will meet everywhere:

Claim	Meaning	Checked by Spring by default?
`iss` (issuer)	who created the token	yes, if you configure an issuer
`sub` (subject)	who the token is about (user or client id)	becomes `Authentication#getName()`
`aud` (audience)	which API(s) the token is meant for	no, you add it (property `audiences` or a validator)
`exp` / `nbf` / `iat`	expires at / not before / issued at (epoch seconds)	`exp` and `nbf` yes, with 60 s clock skew
`jti`	unique token id	no, useful for revocation lists
`scope` / `scp`	granted OAuth scopes (space-separated string or list)	mapped to `SCOPE_x` authorities

Signing: symmetric vs asymmetric

	HMAC (`HS256`)	**RSA / EC (`RS256` , `ES256` , `PS256`)**
Keys	one shared secret signs *and* verifies	the **private** key signs, the **public** key verifies
Who can mint tokens	everybody who can verify	only the key owner
Fits	one service issuing tokens to itself	an identity provider and many APIs (the normal case)

Resource servers download the issuer's public keys as a **JWK Set** (JSON Web Key Set, JWKS) from a URL such as `https://idp.example.com/.well-known/jwks.json`. The `kid` in the token header says which key to use. That is how keys can **rotate** (§11.13).

JWS vs JWE. A signed JWT is a **JWS** (JSON Web Signature). An *encrypted* JWT is a **JWE**, which is rare in APIs. Never put secrets or personal data you would not show the user into a normal JWT, because anyone holding it can decode the payload.

Why tokens

- **No server-side session.** Any instance can verify the signature locally, so APIs scale horizontally without a shared session store.
- **Many parties.** The identity provider issues and the APIs only verify. APIs never see passwords.
- **Delegation.** A token can carry *limited* rights (scopes), for example "read calendar" but not "delete account".

Trade-offs

- **Revocation is hard.** A self-contained JWT stays valid until `exp`, even after logout or a password change. Keep access tokens short-lived (minutes) and use refresh tokens (§11.11), or opaque tokens (§11.7).
- **Size.** Our small token is 670 bytes and is sent on every request. Big claim sets make headers heavy.
- **Theft = access.** A bearer token works for anyone. Always use HTTPS, short lifetimes and safe storage (§11.12).

Self-check

1. *Can a client read the claims of a JWT?* → Yes. The payload is only Base64URL-encoded. The signature protects integrity, not confidentiality.
2. *Why do APIs prefer RS256 over HS256?* → With RS256 the APIs hold only the public key, so a compromised API cannot mint tokens.
3. *What does `kid` do?* → It names the key that signed the token, so the verifier picks the right one from the JWK Set during key rotation.

TL;DR

A JWT is `header.payload.signature`. Claims are readable, and the signature proves origin and integrity. Use asymmetric keys, short lifetimes, and verify `exp`, `iss` and `aud`.

11.2 OAuth 2.0 and OpenID Connect in one picture

What it is

OAuth 2.0 (RFC 6749) is a framework for **delegated authorization**: letting an application act on a user's behalf *without receiving the user's password*. The classic analogy is a hotel key card. Reception (the authorization server) checks your passport once and gives you a card (an access token) that opens *your* room (scopes) until checkout (expiry). The doors (APIs) only check the card.

OpenID Connect (OIDC) is a thin layer on top of OAuth 2.0 that adds **authentication**: an **ID token** (always a JWT) telling the client *who logged in*, a standard `userinfo` endpoint, and discovery metadata at `/.well-known/openid-configuration`.

OAuth role	Who it is	Spring module
Resource owner	the user	—
Client	the app that wants access (web app, SPA backend, mobile app, service)	`spring-security-oauth2-client` (`oauth2Login`, `oauth2Client`)
Authorization server (identity provider, IdP)	issues tokens: Keycloak, Okta, Entra ID, Auth0, Google, Spring Authorization Server	`spring-security-oauth2-authorization-server` (§11.14)
Resource server	the API that accepts tokens	`spring-security-oauth2-resource-server` (`oauth2ResourceServer`)

The tokens

Token	For whom	Format	Lifetime
Access token	the resource server (API)	JWT or opaque	short: 5–60 minutes
Refresh token	the authorization server only, to get new access tokens	usually opaque	hours to weeks, and can be revoked
ID token (OIDC)	the client, to learn who logged in	always a JWT	short, used once at login

Never send an ID token to an API as an access token, and never use an access token to "log a user in" to the client. They have different audiences.

Grant types (how a client gets a token)

Grant	Used by	Status
Authorization code + PKCE	any app with a user: server web apps, SPAs (via BFF), mobile	the standard (§11.9)
Client credentials	service-to-service, no user involved	standard (§11.10)
Refresh token	a client renewing an access token	standard (§11.11)
Device code (RFC 8628)	TVs, CLIs without a browser	standard
Token exchange (RFC 8693)	a service swapping a token for one aimed at another API	growing
Implicit	old SPAs (token in the URL fragment)	**deprecated**, left out of OAuth 2.1
Resource owner password	app collects the user's password	**deprecated**. Spring Security 7 **removed** its client support

The **OAuth 2.1** draft consolidates these best practices: PKCE for every authorization-code client, no implicit and no password grant, exact redirect-URI matching.

Scopes vs roles

A **scope** (`messages.read`) limits what *the client* may do on the user's behalf. The user consented to it. A **role** is about what *the user* may do in your system. A token can carry both, and your API decides how to combine them. Spring maps scopes to `SCOPE_x` authorities by default (§11.5).

TL;DR

OAuth 2.0 = delegated authorization with access tokens. OIDC adds login with ID tokens. Use authorization code + PKCE for users and client credentials for services. Implicit and password grants are obsolete.

11.3 A JWT resource server in Spring

What it is

A **resource server** is an API that accepts access tokens. With `spring-boot-starter-security-oauth2-resource-server` (Boot 4's name; the old `spring-boot-starter-oauth2-resource-server` is deprecated), one DSL call adds a `BearerTokenAuthenticationFilter` to the chain:

```
http.oauth2ResourceServer(rs -> rs.jwt(Customizer.withDefaults()));
```

With a real identity provider, one property is usually all you need:

```
# Spring reads https://idp.example.com/.well-known/openid-configuration (or oauth-authorization-server)
# to find the JWK Set URL, and validates that iss equals this value
spring.security.oauth2.resourceserver.jwt.issuer-uri=https://idp.example.com/realms/demo
# optional extras
spring.security.oauth2.resourceserver.jwt.audiences=notes-api
spring.security.oauth2.resourceserver.jwt.jws-algorithms=RS256
# alternatives: jwk-set-uri=... (no discovery) or public-key-location=classpath:key.pub (static key)
```

In our project there is no IdP, so we declare the `JwtDecoder` bean ourselves with a locally generated key pair. A `JwtDecoder` bean always takes precedence over the properties.

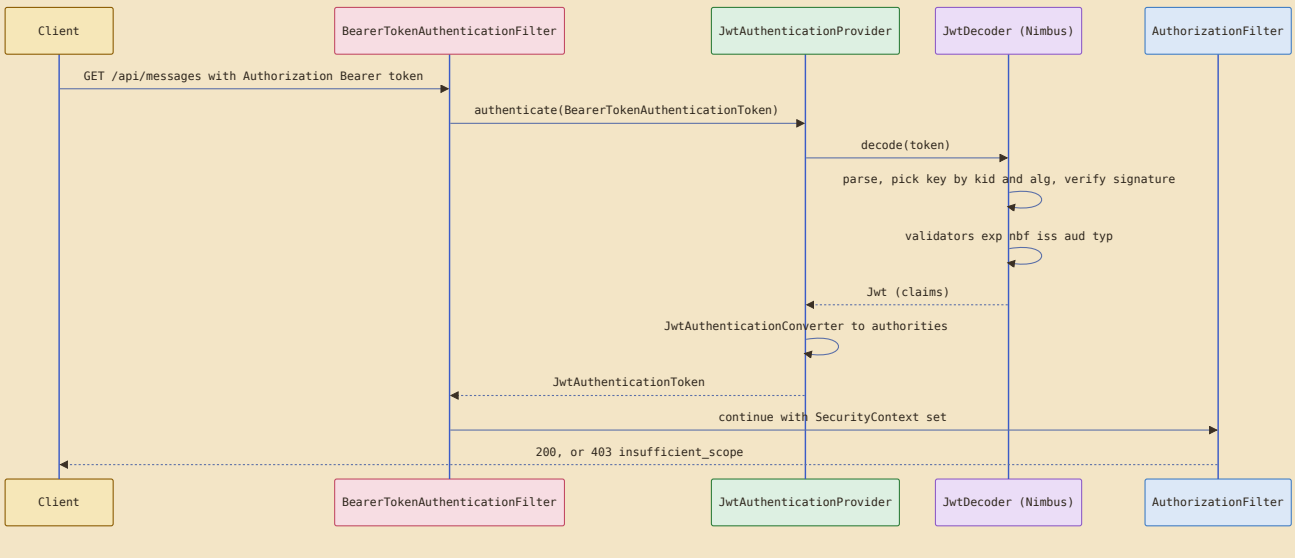
```
@Bean
KeyPair signingKeys() throws Exception {                    // demo only: new keys at each start
    KeyPairGenerator gen = KeyPairGenerator.getInstance("RSA");
    gen.initialize(2048);
    return gen.generateKeyPair();
}

@Bean
JwtDecoder jwtDecoder(KeyPair keys, @Value("${demo.jwt.issuer}") String issuer,
                      @Value("${demo.jwt.audience}") String audience) {
    NimbusJwtDecoder decoder = NimbusJwtDecoder.withPublicKey((RSAPublicKey)
keys.getPublic()).build();
    OAuth2TokenValidator<Jwt> audienceCheck =
            new JwtClaimValidator<List<String>>("aud", aud -> aud != null && aud.contains(audience));
    decoder.setJwtValidator(new DelegatingOAuth2TokenValidator<>(
            JwtValidators.createDefaultWithIssuer(issuer), audienceCheck));
    return decoder;
}
```

The API chain:

```
@Bean @Order(2)
SecurityFilterChain api(HttpSecurity http, JwtAuthenticationConverter converter) throws Exception {
    http.authorizeHttpRequests(a -> a
            .requestMatchers(HttpMethod.GET, "/api/messages").hasAuthority("SCOPE_messages.read")
            .requestMatchers(HttpMethod.POST, "/api/messages").hasAuthority("SCOPE_messages.write")
            .requestMatchers("/api/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated())
        .oauth2ResourceServer(rs -> rs.jwt(jwt -> jwt.jwtAuthenticationConverter(converter)))
        .csrf(c -> c.disable())                // no cookies -> no CSRF risk
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
    return http.build();
}
```

How it works



1. `BearerTokenAuthenticationFilter` takes the token from `Authorization: Bearer ...`. Form parameters and query strings are off by default, because tokens in URLs leak into logs.

2. `JwtAuthenticationProvider` calls the **`JwtDecoder`**. Nimbus parses the token, rejects algorithms it wasn't configured for (the classic `alg: none` trick fails here), and **verifies the signature**.
3. **Validators** run: `JwtTimestampValidator` (`exp`/`nbf` with 60 s clock skew), the issuer validator if configured, your own (audience), and in Spring Security 7 a `JwtTypeValidator` that accepts `typ` = `JWT` or no `typ` at all. All `JwtValidators.createDefault...()` methods include it now.
4. A **`JwtAuthenticationConverter`** turns claims into authorities (§11.5) and builds a `JwtAuthenticationToken`. Controllers can take `@AuthenticationPrincipal Jwt jwt`.
5. Any failure → **401** with a `WWW-Authenticate: Bearer error="invalid_token"` header (RFC 6750). A valid token without the needed authority → **403** with `error="insufficient_scope"`.

Proved: status codes with real tokens

Nothing is mocked here. The test obtains real tokens from `/token` and calls the API:

```
no token                          401 WWW-Authenticate: Bearer
resource_metadata="http://localhost/.well-known/oauth-protected-resource"
alice GET (messages.read)         200 ["welcome"]
alice POST (needs messages.write) 403 WWW-Authenticate: Bearer error="insufficient_scope",
error_description="The request requires higher privileges than provided by the access token.",
error_uri="https://tools.ietf.org/html/rfc6750#section-3.1"
bob POST                          201 added by bob
bob GET admin (roles=[ADMIN])     200 audit log for bob
garbage token                     401 WWW-Authenticate: Bearer error="invalid_token",
error_description="An error occurred while attempting to decode the Jwt: Malformed token",
error_uri="https://tools.ietf.org/html/rfc6750#section-3.1",
resource_metadata="http://localhost/.well-known/oauth-protected-resource"
```

The `resource_metadata` parameter points to the **OAuth 2.0 Protected Resource Metadata** document (RFC 9728), which the resource server publishes on its own. A client that knows nothing about the API can discover which authorization server to ask:

```
GET /.well-known/oauth-protected-resource -> 200
{"resource":"http://localhost","bearer_methods_supported":
["header"],"tls_client_certificate_bound_access_tokens":true}
```

With a real issuer configured, the document also lists `authorization_servers`. Customise it with `oauth2ResourceServer(rs -> rs.protectedResourceMetadata(...))`. The Model Context Protocol's authorization spec, for example, relies on this discovery step.

Proved: every validation rule

The test mints tokens with deliberate defects:

```
String expiredLongAgo = mint(encoder, c ->
c.issuedAt(now.minusSeconds(3600)).expiresAt(now.minusSeconds(600)));
String expiredJustNow = mint(encoder, c ->
c.issuedAt(now.minusSeconds(600)).expiresAt(now.minusSeconds(30)));
String wrongIssuer    = mint(encoder, c -> c.issuer("https://evil.test"));
String wrongAudience  = mint(encoder, c -> c.audience(List.of("billing-api")));
String otherKey       = mint(attacker, c -> {});              // attacker's own RSA key pair
// valid token, payload swapped for one with more scopes, original signature kept
String tampered = parts[0] + "." + forgedPayload + "." + parts[2];
```

Output (the `error_uri` and `resource_metadata` parameters are trimmed as `...`):

```
valid                              200 ["welcome","hi"]
expired 10 min ago                 401 WWW-Authenticate: Bearer error="invalid_token",
error_description="An error occurred while attempting to decode the Jwt: Jwt expired at 2026-09-
28T23:19:21Z", ...
expired 30 s ago (clock skew 60 s) 200 ["welcome","hi"]
wrong issuer                       401 WWW-Authenticate: Bearer error="invalid_token",
error_description="An error occurred while attempting to decode the Jwt: The iss claim is not valid",
...
wrong audience                     401 WWW-Authenticate: Bearer error="invalid_token",
error_description="An error occurred while attempting to decode the Jwt: The aud claim is not valid",
...
signed with another key            401 WWW-Authenticate: Bearer error="invalid_token",
error_description="An error occurred while attempting to decode the Jwt: Signed JWT rejected: Invalid
signature", ...
payload edited after signing       401 WWW-Authenticate: Bearer error="invalid_token",
error_description="An error occurred while attempting to decode the Jwt: Signed JWT rejected: Invalid
signature", ...
```

What to take away:

- A token that expired **30 s ago still passes**. `JwtTimestampValidator` allows **60 seconds of clock skew** between servers. Remember this when you test expiry.
- **Changing one character of the payload breaks the signature.** That is why claims can be trusted even though anyone can read them.
- **Audience matters.** A token issued for `billing-api` by the *same* trusted issuer is refused. Without an audience check, any token from your IdP, meant for any API, would be accepted by every API.

Common mistakes

- **Skipping the audience check.** Set `spring.security.oauth2.resourceserver.jwt.audiences` or add a validator.
- **Replacing** the validators with only your own. `decoder.setJwtValidator(audienceCheck)` alone drops the expiry check! Always combine with `JwtValidators.createDefault...()`, as above.
- Putting the resource server behind session-based security (`IF_REQUIRED`), so a session cookie quietly becomes a second way in. Use `STATELESS` and disable CSRF only on the token chain.
- Logging full tokens. Log `jti` or `sub` instead.

TL;DR

`oauth2ResourceServer(jwt)` + `issuer-uri` (or your own `JwtDecoder`). Spring verifies the signature, `exp`/`nbf` (60 s skew), `iss`, `typ`, and your `aud` rule. Bad or missing token → 401 `invalid_token`. Missing authority → 403 `insufficient_scope`.

11.4 Minting tokens with `NimbusJwtEncoder`

What it is

`JwtEncoder` is the counterpart of `JwtDecoder`: it signs claims into a JWT. Spring Security 7 added builder methods to `NimbusJwtEncoder`, so creating one from a key pair or secret is a one-liner (before, you had to build a Nimbus `JWKSource` by hand):

```
@Bean
JwtEncoder jwtEncoder(KeyPair keys) {
    return NimbusJwtEncoder.withKeyPair((RSAPublicKey) keys.getPublic(), (RSAPrivateKey)
keys.getPrivate())
            .jwkPostProcessor(jwk -> jwk.keyID("demo-key-1"))   // becomes "kid" in the header
            .build();
}
// also: withKeyPair(ECPublicKey, ECPrivateKey) for ES256, withSecretKey(SecretKey) for HS256
```

Our `/token` endpoint is protected by HTTP Basic (its own chain, `@Order(1)`). It turns the logged-in user's authorities into claims:

```
@PostMapping("/token")
TokenResponse token(Authentication auth) {
    Instant now = Instant.now();
    Duration ttl = Duration.ofMinutes(15);
    List<String> authorities = auth.getAuthorities().stream().map(GrantedAuthority::getAuthority)
            .filter(a -> !a.startsWith("FACTOR_")).toList();          // drop Security 7 MFA factors
    String scope = String.join(" ", authorities.stream().filter(a ->
!a.startsWith("ROLE_")).toList());
    List<String> roles = authorities.stream().filter(a -> a.startsWith("ROLE_")).map(a ->
a.substring(5)).toList();
    JwtClaimsSet claims = JwtClaimsSet.builder()
            .issuer(issuer).audience(List.of(audience)).subject(auth.getName())
            .issuedAt(now).expiresAt(now.plus(ttl)).id(UUID.randomUUID().toString())
            .claim("scope", scope).claim("roles", roles)
            .build();
    JwsHeader header = JwsHeader.with(SignatureAlgorithm.RS256).type("JWT").build();
    String jwt = encoder.encode(JwtEncoderParameters.from(header, claims)).getTokenValue();
    return new TokenResponse(jwt, "Bearer", ttl.toSeconds());
}
```

Note the `FACTOR_` filter. After password login, Spring Security 7 adds `FACTOR_PASSWORD` to the authorities (10.4). It describes *how* the user logged in and does not belong in a scope claim.

When to issue your own tokens — and when not

Situation	Self-issued JWT (`JwtEncoder`)	Real authorization server
tests, demos, this book	ideal: no infrastructure	overkill
one internal service issuing short tokens to its own clients	acceptable	better long-term
users from several apps, SSO, social login, MFA, consent, refresh-token rotation, revocation	**no**, you would be re-writing an IdP	**yes**: Keycloak, a cloud IdP, or Spring Authorization Server (§11.14)

Writing your own login-and-token server means owning password storage, lockout, MFA, key rotation, refresh tokens and revocation forever. Use an existing authorization server unless the need is truly tiny.

> **TL;DR**
>
> `NimbusJwtEncoder.withKeyPair(...)` / `withSecretKey(...)` (Spring Security 7) + `JwtClaimsSet` + `JwsHeader` = a signed token in a few lines. Great for tests and demos. For real users, use an authorization server.

11.5 Mapping claims to authorities

What it is

The decoder produces claims. Authorization rules need **authorities**. By default `JwtAuthenticationConverter` uses a **`JwtGrantedAuthoritiesConverter`** that reads the `scope` (or `scp`) claim, splits it on spaces and prefixes each value with `SCOPE_`. So `scope: "messages.read messages.write"` becomes `SCOPE_messages.read`, `SCOPE_messages.write`. Nothing else in the token becomes an authority unless you say so.

IdPs put roles in different places: Keycloak uses `realm_access.roles`, Entra ID uses `roles`, Okta uses `groups`. Our token carries a `roles` claim, so the project combines two converters:

```
public class ScopeAndRoleAuthorities implements Converter<Jwt, Collection<GrantedAuthority>> {
    private final JwtGrantedAuthoritiesConverter scopes = new JwtGrantedAuthoritiesConverter(); //
scope/scp -> SCOPE_
    private final JwtGrantedAuthoritiesConverter roles = new JwtGrantedAuthoritiesConverter();

    public ScopeAndRoleAuthorities() {
        roles.setAuthoritiesClaimName("roles");
        roles.setAuthorityPrefix("ROLE_");
    }

    @Override
    public Collection<GrantedAuthority> convert(Jwt jwt) {
        Collection<GrantedAuthority> all = new ArrayList<>(scopes.convert(jwt));
        all.addAll(roles.convert(jwt));
        return all;
    }
}

@Bean
JwtAuthenticationConverter jwtAuthenticationConverter(ScopeAndRoleAuthorities authorities) {
    JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
    converter.setJwtGrantedAuthoritiesConverter(authorities);
    converter.setPrincipalClaimName("sub");                // Authentication#getName() = sub claim
    return converter;
}
```

This is why `bob GET admin (roles=[ADMIN])` returned 200: `roles: ["ADMIN"]` became `ROLE_ADMIN`, and `hasRole("ADMIN")` matched.

If one claim is enough, Boot can configure the default converter with properties and no code:

```
spring.security.oauth2.resourceserver.jwt.authorities-claim-name=roles
spring.security.oauth2.resourceserver.jwt.authority-prefix=ROLE_
spring.security.oauth2.resourceserver.jwt.principal-claim-name=preferred_username
```

Common mistakes

- Checking `hasRole("messages.read")` or `hasAuthority("messages.read")` for a scope. The authority is `SCOPE_messages.read`.
- Nested claims (`realm_access.roles`). `JwtGrantedAuthoritiesConverter` reads top-level claims only, so write a small converter that walks the map.
- Using `sub` for display. With many IdPs `sub` is an opaque id, so read `preferred_username` or `email` for humans, but keep keying data by `sub` (+ `iss`).

TL;DR

Default: `scope` → `SCOPE_x`. Anything else (roles, groups) needs a custom converter or the Boot `authorities-claim-name` / `authority-prefix` properties.

11.6 Testing token security

Two ways to test

Approach	How	Proves
Real tokens	mint with `JwtEncoder` (or call `/token`), send `Authorization: Bearer ...`	decoder, validators, converter *and* rules. Everything in §11.3 was tested this way
`jwt()` post-processor	`mvc.get().uri(..).with(jwt()...)`	only the **rules**. Decoding and validation are skipped, and a ready-made `JwtAuthenticationToken` is placed in the context

Use real tokens for a handful of tests around the decoder configuration (issuer, audience, expiry). Use `jwt()` for the many rule tests, since it is faster and needs no keys.

The `jwt()` pitfall — proved

```
var plain = mvc.get().uri("/api/messages").with(jwt());                  // sub=user, scope=read
var explicit = mvc.post().uri("/api/messages").contentType(MediaType.TEXT_PLAIN).content("x")
        .with(jwt().jwt(j -> j.subject("carol")).authorities(new
SimpleGrantedAuthority("SCOPE_messages.write")));
var scopeClaim = mvc.get().uri("/api/messages").with(jwt().jwt(j -> j.claim("scope",
"messages.read")));
// PITFALL: jwt() does not use your JwtAuthenticationConverter unless you pass it in
var rolesIgnored = mvc.get().uri("/api/admin/audit").with(jwt().jwt(j -> j.claim("roles",
List.of("ADMIN"))));
var rolesMapped  = mvc.get().uri("/api/admin/audit")
        .with(jwt().jwt(j -> j.claim("roles", List.of("ADMIN"))).authorities(appConverter));
```

Output:

```
jwt() default, GET messages             -> 403
jwt() SCOPE_messages.write, POST        -> 201 added by carol
jwt() scope claim messages.read, GET    -> 200
jwt() roles=[ADMIN], default converter  -> 403
jwt() roles=[ADMIN], app converter      -> 200
```

1. `jwt()` alone gives subject `user` and scope `read`, so the authority is `SCOPE_read`, not `SCOPE_messages.read`. That's a 403.
2. `.authorities(...)` sets the authorities directly, whatever the claims say.
3. Claims are mapped with the **default** `JwtGrantedAuthoritiesConverter`, so `scope` works…
4. …but your custom `roles` mapping is **not applied**. The test gets 403 although the same token from a real IdP would get 200. Pass your converter with `.authorities(converter)`, or test that path with a real token.

The same helpers exist for the other token types: `opaqueToken()` (§11.7), `oauth2Login()` and `oidcLogin()` (§11.9), and `oauth2Client()` for an authorized client in tests. With `@WithMockUser` you can test URL rules too, but the principal is then not a `Jwt`, so controllers taking `@AuthenticationPrincipal Jwt` receive `null`.

TL;DR

Test the decoder with real, locally signed tokens. Test rules with `jwt()`, and hand it your authorities converter or you are testing a different mapping from production.

11.7 Opaque tokens and introspection

What it is

An **opaque token** is a random string with no readable content (`tok-alice-7f3a`). The resource server cannot verify it alone. It asks the authorization server's **introspection endpoint** (RFC 7662) "is this token active, and for whom?" on every request (with caching, if you add it):

```
POST /oauth2/introspect   token=tok-alice-7f3a   (client authenticates with its own id/secret)
-> {"active": true, "sub": "alice", "scope": "notes.read", "exp": 1790000000, ...}
```

```
spring.security.oauth2.resourceserver.opaquetoken.introspection-
uri=https://idp.example.com/oauth2/introspect
spring.security.oauth2.resourceserver.opaquetoken.client-id=notes-api
spring.security.oauth2.resourceserver.opaquetoken.client-secret=${INTROSPECTION_SECRET}
```

With those properties Boot creates an `OpaqueTokenIntrospector` that calls the endpoint (classic implementation: `SpringOpaqueTokenIntrospector`). Spring Security 7.1 adds `RestClientOpaqueTokenIntrospector`, built on `RestClient`. To stay offline, our project supplies its own `OpaqueTokenIntrospector` that answers from a map, in the same shape the real endpoint would:

```
class FakeIntrospector implements OpaqueTokenIntrospector {
    private final Map<String, Map<String, Object>> activeTokens = Map.of(
            "tok-alice-7f3a", Map.of("active", true, "sub", "alice", "scope", "notes.read", "exp",
Instant.now().plusSeconds(600)),
            "tok-bob-91c2", Map.of("active", true, "sub", "bob", "scope", "notes.read notes.write",
"exp", Instant.now().plusSeconds(600)));

    @Override
    public OAuth2AuthenticatedPrincipal introspect(String token) {
        Map<String, Object> claims = activeTokens.get(token);
        if (claims == null) throw new BadOpaqueTokenException("Provided token isn't active"); //
{"active":false}
        List<GrantedAuthority> authorities = Arrays.stream(((String) claims.get("scope")).split(" "))
                .map(s -> (GrantedAuthority) new SimpleGrantedAuthority("SCOPE_" + s)).toList();
        return new OAuth2IntrospectionAuthenticatedPrincipal((String) claims.get("sub"), claims,
authorities);
    }
}

// chain for /opaque/**
http.securityMatcher("/opaque/**")
    .authorizeHttpRequests(a -> a
        .requestMatchers("/opaque/notes/write").hasAuthority("SCOPE_notes.write")
        .anyRequest().hasAuthority("SCOPE_notes.read"))
    .oauth2ResourceServer(rs -> rs.opaqueToken(o -> o.introspector(introspector)));
```

Output:

```
alice GET   -> 200 notes for alice (scope=notes.read)
alice write -> 403
bob write   -> 200 written by bob
inactive    -> 401 Bearer error="invalid_token", error_description="Provided token isn't active", ...
opaqueToken() -> 200 notes for carol (scope=notes.read)
```

The principal is an `OAuth2AuthenticatedPrincipal` (not a `Jwt`), and the last line shows the `opaqueToken()` test post-processor.

JWT vs opaque

	JWT (self-contained)	**Opaque + introspection**
Validation	local, just a signature check	network call to the IdP per request (or per cache period)
Revocation	only when it expires (or via a deny-list)	**immediate**: the IdP answers `active: false`
Latency / IdP load	minimal	higher, and the IdP must be highly available
Privacy	claims readable by any holder	nothing readable
Typical use	APIs inside your platform	high-risk operations, tokens handed to third parties

A common hybrid is **opaque tokens outside, JWTs inside**: the API gateway introspects the opaque token and forwards a short-lived JWT to internal services (the "phantom token" pattern).

TL;DR

Opaque tokens are meaningless strings that the API checks with the IdP (RFC 7662). You get instant revocation and privacy, and pay in latency and IdP load. `oauth2ResourceServer(rs -> rs.opaqueToken(...))`.

11.8 Multi-tenancy: several issuers in one API

What it is

A **multi-tenant** API serves several customers ("tenants"), each with its *own* identity provider or realm, and so its own issuer and keys. The token's `iss` claim tells you which tenant it claims to come from. The API must pick **that tenant's keys** to verify it, and reject issuers it does not trust.

Spring's `JwtIssuerAuthenticationManagerResolver` reads `iss` *without trusting it yet*, looks up an `AuthenticationManager` for that issuer, and lets that manager verify the signature with the issuer's keys:

```
Map<String, AuthenticationManager> managers = new ConcurrentHashMap<>();
JwtIssuerAuthenticationManagerResolver resolver = new JwtIssuerAuthenticationManagerResolver(issuer ->
{
    if (!keys.isTrusted(issuer)) return null;         // unknown issuer -> 401
    return managers.computeIfAbsent(issuer, iss -> {
        NimbusJwtDecoder decoder = NimbusJwtDecoder.withPublicKey(keys.publicKey(iss)).build();
        decoder.setJwtValidator(JwtValidators.createDefaultWithIssuer(iss));
        return new ProviderManager(new JwtAuthenticationProvider(decoder));
    });
});
http.securityMatcher("/tenant/**")
    .authorizeHttpRequests(a -> a.anyRequest().authenticated())
    .oauth2ResourceServer(rs -> rs.authenticationManagerResolver(resolver));
```

Output:

```
tenant A token            -> 200 ann from https://idp.tenant-a.test
tenant B token            -> 200 ben from https://idp.tenant-b.test
B-signed, iss = tenant A  -> 401
untrusted issuer          -> 401
```

The third line is the important one. A token **signed by tenant B's key that claims to be from tenant A** is rejected, because the resolver verified it with tenant A's key. With real IdPs that publish discovery metadata, one line does the same job: `JwtIssuerAuthenticationManagerResolver.fromTrustedIssuers("https://idp.tenant-a.test", "https://idp.tenant-b.test")`. It discovers each issuer's JWK Set on first use.

Common mistakes

- Building a decoder for **any** `iss` value the token names ("dynamic issuer" without an allow-list). An attacker then runs their own IdP and names it. Always check against a trusted list.
- Forgetting tenant isolation *after* authentication. The token proves the user belongs to tenant A. Your queries must still filter by tenant.

TL;DR

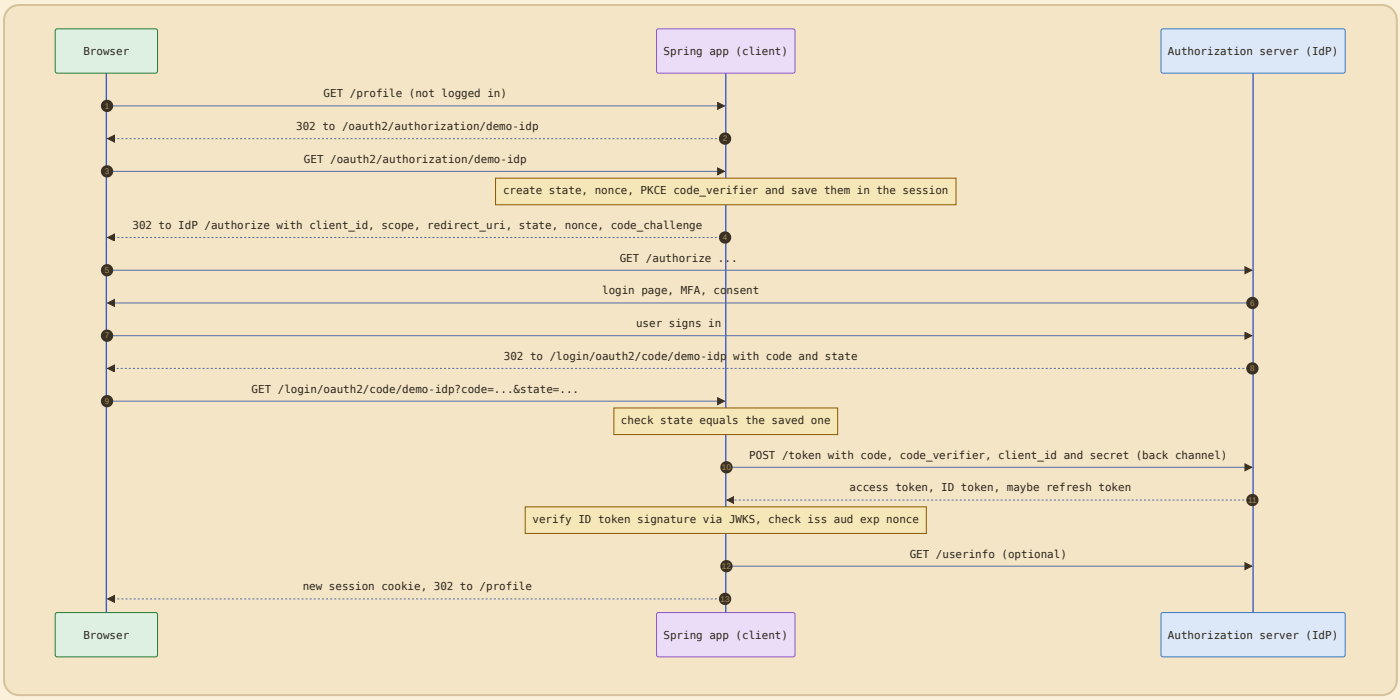
Several issuers → `JwtIssuerAuthenticationManagerResolver` with an allow-list. The `iss` claim chooses the keys, and the signature must match that issuer's key.

11.9 OAuth2 login: "Sign in with ..."

What it is

OAuth2 login lets users sign in to *your web app* with an account at an identity provider (Google, GitHub, Keycloak, your company's Entra ID). Your app is the **client**. It never sees the password. It receives an **authorization code**, exchanges it for tokens *server-side*, and creates a normal session for the browser. This is the **authorization-code flow**, and with OIDC it also yields an ID token.

How it works



The protective parameters:

Parameter	Protects against	How
`state`	CSRF on the callback: an attacker making your browser finish *their* login	random value stored in the session and compared on return
`nonce` (OIDC)	replay of an old ID token	random value that must come back inside the ID token
PKCE (`code_challenge` + `code_verifier`, RFC 7636)	a stolen authorization code	the app sends `SHA-256(verifier)` first, and only the holder of the verifier can redeem the code
exact **`redirect_uri`**	codes sent to an attacker's site	the IdP only redirects to pre-registered URIs
code exchange on the **back channel**	tokens exposed to the browser	tokens travel server-to-server, and the browser only gets a session cookie

Configuration

With a real OIDC provider, Boot needs just the client credentials and the issuer:

```
spring.security.oauth2.client.registration.demo-idp.client-id=notes-web
spring.security.oauth2.client.registration.demo-idp.client-secret=${DEMO_IDP_SECRET}
spring.security.oauth2.client.registration.demo-idp.scope=openid,profile,email
spring.security.oauth2.client.provider.demo-idp.issuer-uri=https://idp.example.test
```

Boot has built-in provider defaults for `google`, `github`, `facebook` and `okta` (for those, the registration id is enough). Our project spells out every endpoint instead of `issuer-uri`, so it starts without contacting anyone:

```
spring.security.oauth2.client.registration.demo-idp.authorization-grant-type=authorization_code
spring.security.oauth2.client.registration.demo-idp.redirect-uri=
{baseUrl}/login/oauth2/code/{registrationId}
spring.security.oauth2.client.provider.demo-idp.authorization-
uri=https://idp.example.test/oauth2/authorize
spring.security.oauth2.client.provider.demo-idp.token-uri=https://idp.example.test/oauth2/token
spring.security.oauth2.client.provider.demo-idp.jwk-set-uri=https://idp.example.test/oauth2/jwks
spring.security.oauth2.client.provider.demo-idp.user-info-uri=https://idp.example.test/userinfo
spring.security.oauth2.client.provider.demo-idp.user-name-attribute=sub
```

```
http.authorizeHttpRequests(a -> a.requestMatchers("/",
"/public/**").permitAll().anyRequest().authenticated())
    .oauth2Login(Customizer.withDefaults())
    .logout(Customizer.withDefaults());

@GetMapping("/profile")
String profile(@AuthenticationPrincipal OidcUser user) {
    return "hello " + user.getFullName() + " <" + user.getEmail() + ">, sub=" + user.getSubject()
            + ", issuer=" + user.getIssuer();
}
```

What we can run without an IdP

Steps 1–4 of the diagram (everything before the browser leaves our app) and the callback's `state` check run for real. The user signing in at the IdP and the code exchange are not run, since there is no IdP. The `oidcLogin()` post-processor then simulates a finished login:

```
1. GET /profile (anonymous)       -> 302 /oauth2/authorization/demo-idp
2. GET /oauth2/authorization/demo-idp -> 302 to https://idp.example.test/oauth2/authorize
     client_id=notes-web
     code_challenge=<random>
     code_challenge_method=S256
     nonce=<random>
     redirect_uri=http://localhost/login/oauth2/code/demo-idp
     response_type=code
     scope=openid%20profile%20email
     state=<random>
   session attribute saved: true
3. callback with forged state      -> 302 /login?error
oidcLogin() GET /profile -> 200 hello Jane Doe <jane@example.com>, sub=248289761001,
issuer=https://idp.example.test
```

- The authorization request contains `state`, `nonce` **and PKCE** (`code_challenge`, `S256`) even though this client has a secret. Spring Security 7 uses PKCE for confidential clients too. If an old IdP can't handle it, turn it off for that registration by building it with `.clientSettings(ClientRegistration.ClientSettings.builder().requireProofKey(false).build())`.
- A callback whose `state` was never issued to this browser session is refused (`/login?error`). This is the CSRF protection of the login flow.
- After a real login the principal is an **`OidcUser`** (ID-token claims + userinfo). The tokens are stored server-side as an `OAuth2AuthorizedClient`, and the browser only holds the session cookie. So OAuth2 login is **session-based**: keep CSRF protection on (Chapter 10).

OAuth2 vs OIDC login

With the `openid` scope Spring performs **OIDC** login: it validates the ID token and builds an `OidcUser`. Without it (for example GitHub, which is plain OAuth2) Spring calls the provider's user-info endpoint and builds an `OAuth2User` from the JSON. Mapping external users to *your* accounts (first

login = sign-up, linking by verified e-mail, local roles) is your job. Do it in a custom `OidcUserService` / `OAuth2UserService`, or in a listener for the login success event.

TL;DR

`oauth2Login()` + a client registration = "Sign in with X" via authorization code, `state`, `nonce` and PKCE. Tokens stay on the server, and the browser gets a session. Test with `oidcLogin()` / `oauth2Login()`.

11.10 Calling other services: client credentials and token relay

What it is

Your service often has to call *another* protected API. There are two cases:

Case	Whose token?	Grant
a nightly job calls the reports API	the **service's own** identity	**client credentials**: the service authenticates with its client id + secret (or a private-key JWT / mTLS) and gets an access token
a user's request fans out to downstream APIs	the **user's** identity	**token relay**: forward the user's access token, or better, exchange it (RFC 8693) for one aimed at the downstream API

How it looks in Spring (not run: needs a token endpoint)

Configure a registration and let `OAuth2AuthorizedClientManager` fetch, cache and renew tokens:

```
spring.security.oauth2.client.registration.reports.provider=demo-idp
spring.security.oauth2.client.registration.reports.client-id=nightly-job
spring.security.oauth2.client.registration.reports.client-secret=${REPORTS_SECRET}
spring.security.oauth2.client.registration.reports.authorization-grant-type=client_credentials
spring.security.oauth2.client.registration.reports.scope=reports.read
```

```
// RestClient (Chapter 6) with an interceptor that adds "Authorization: Bearer <token>"
@Bean
RestClient reportsClient(RestClient.Builder builder, OAuth2AuthorizedClientManager manager) {
    return builder.baseUrl("https://reports.internal")
            .requestInterceptor(new OAuth2ClientHttpRequestInterceptor(manager))
            .build();
}

String csv = reportsClient.get().uri("/daily")
        .attributes(clientRegistrationId("reports"))   // which registration's token to use
        .retrieve().body(String.class);
```

With **HTTP interfaces** (Chapter 6), Spring Security 7 lets you put `@ClientRegistrationId` on the whole interface. Register an `OAuth2RestClientHttpServiceGroupConfigurer` bean so the HTTP service clients get the interceptor:

```java
@HttpExchange("/reports")
@ClientRegistrationId("reports")           // type-level support is new in Spring Security 7
interface ReportsApi {
    @GetExchange("/daily") String daily();
}

@Bean
OAuth2RestClientHttpServiceGroupConfigurer securityConfigurer(OAuth2AuthorizedClientManager manager) {
    return OAuth2RestClientHttpServiceGroupConfigurer.from(manager);
}
```

The interceptor asks the manager for a token each time. The manager returns the cached one while it is valid, and requests or refreshes one when needed. You never parse expiry yourself.

Token relay. In a resource server, the incoming token is available as `JwtAuthenticationToken#getToken().getTokenValue()`, and an interceptor can copy it to outgoing calls. In **Spring Cloud Gateway**, the `TokenRelay` filter does this for logged-in users (Chapter 16). Forwarding the same token to every service means every service must accept the same audience. **Token exchange** (supported by Spring Security's OAuth2 client since 6.3) gets a new, narrower token per downstream API instead.

TL;DR

Services call APIs with **client credentials**, via `OAuth2AuthorizedClientManager` + `OAuth2ClientHttpRequestInterceptor` on `RestClient` (or `@ClientRegistrationId` on HTTP interfaces). User context flows by token relay or, better, token exchange.

11.11 Refresh tokens, lifetimes, revocation and logout

What it is

Short-lived access tokens limit damage, but users shouldn't have to log in every 10 minutes. A **refresh token** is a long-lived credential that the client sends **only to the authorization server** to get a new access token without user interaction.

Practice	Why
access token 5–15 min, refresh token hours/days	a stolen access token is soon useless
refresh-token rotation: every use returns a *new* refresh token and invalidates the old one	a stolen refresh token used twice is detected, and the IdP revokes the whole family
bind refresh tokens to the client (secret, DPoP or mTLS)	a stolen token alone is not enough
never give refresh tokens to browser JavaScript	XSS could steal a long-lived credential (§11.12)

In Spring, the resource server never sees refresh tokens. The **client side** handles them: the `OAuth2AuthorizedClientManager` behind `oauth2Login` and the interceptor above refreshes an expired access token automatically when a refresh token is present.

Revocation and logout

- **Revoking** a token (RFC 7009) tells the IdP it's dead. Opaque tokens stop working at once. JWTs keep working until `exp`, unless your APIs check a deny-list of `jti` values (usually not worth the cost if lifetimes are short).

- **Logging out** of your app (`POST /logout`) ends *your* session but not the user's session at the IdP. The next "Sign in with ..." logs them straight back in. For single logout, configure **OIDC RP-initiated logout** so Spring also redirects to the IdP's `end_session_endpoint`:

```
http.logout(l -> l.logoutSuccessHandler(new
OidcClientInitiatedLogoutSuccessHandler(clientRegistrationRepository)));
```

Spring Security also supports **OIDC back-channel logout** (`http.oidcLogout(...)`), where the IdP calls your app to end sessions when the user logs out elsewhere.

TL;DR

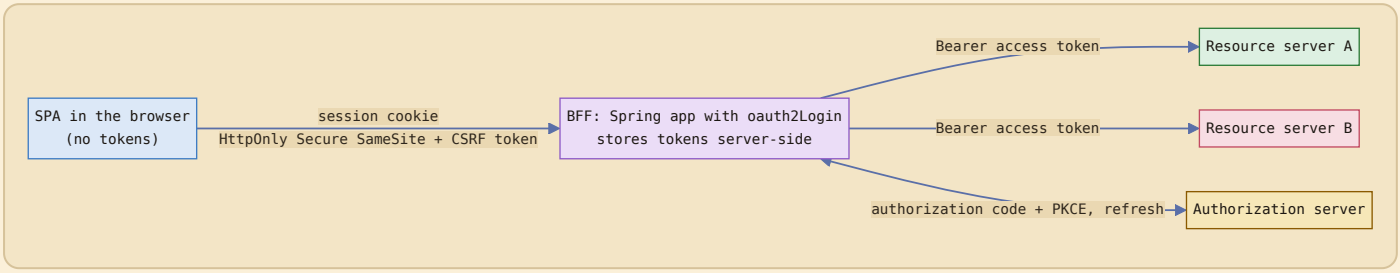
Short access tokens + rotating refresh tokens, handled by the client-side `OAuth2AuthorizedClientManager`. Revocation is instant only for opaque tokens. Use OIDC RP-initiated logout to end the IdP session too.

11.12 Where should a browser keep tokens? The BFF pattern

The problem it solves

A single-page app that holds tokens in JavaScript has to store them somewhere. `localStorage` and `sessionStorage` are readable by **any script on the page**, so one XSS bug leaks every user's tokens (see 14.3 Web Storage and 14.10 XSS). In-memory storage is lost on reload and is still reachable by injected script.

The recommended answer: Backend for Frontend (BFF)



1. The SPA talks only to its **BFF** (Backend for Frontend) on the same site, using an `HttpOnly` session cookie that JavaScript cannot read.
2. The BFF is a confidential OAuth2 client (`oauth2Login`, §11.9). It keeps access and refresh tokens server-side and refreshes them.
3. The BFF forwards API calls with the bearer token (token relay, §11.10). Spring Cloud Gateway with `TokenRelay` is a common BFF.
4. Because the SPA now uses cookies, **CSRF protection is required**. Use `csrf.spa()` (10.8).

The IETF draft *OAuth 2.0 for Browser-Based Applications* recommends this architecture as the most secure option. See also the BFF pattern notes.

Where tokens live	XSS steals tokens?	CSRF risk	Complexity
`localStorage`	**yes**	no	low
JS memory only	harder, still possible while the page runs	no	low, but lost on reload
BFF (`HttpOnly` cookie session)	no, the tokens never reach the browser	yes, so use CSRF tokens	an extra backend

TL;DR

Don't keep long-lived tokens in browser JavaScript. Put a BFF in front: the browser gets an `HttpOnly` session cookie (plus CSRF protection), and the tokens stay on the server.

11.13 Storing keys and secrets

Passwords are covered in 10.5: hash them with `DelegatingPasswordEncoder`. Token systems add more secrets:

Secret	Where it lives	Rules
JWT signing private key	the authorization server only: a vault, KMS or HSM	never in Git, never in a resource server. Resource servers get *public* keys from the JWKS URL
Client secret (confidential clients)	environment variable / secret manager, injected as `${...}` like `${DEMO_IDP_SECRET}` above	rotate it, and prefer `private_key_jwt` or mTLS client authentication for high-value clients
Refresh tokens stored by a BFF	server-side session or database, encrypted at rest	never sent to the browser
HMAC secret (if you use HS256)	secret manager	at least 256 bits of randomness. Everyone who can verify can also forge

Key rotation without downtime:

1. Publish the *new* public key in the JWKS next to the old one (a different `kid`).
2. Wait longer than the verifiers' JWKS cache time. `NimbusJwtDecoder` caches the JWK Set and re-fetches it when it meets an unknown `kid`.
3. Start signing with the new key.
4. Remove the old key after the longest token lifetime has passed.

Our demo generates a fresh key pair at every start. Every issued token becomes invalid on restart, and several instances would disagree. That's fine for tests and wrong for production.

11.14 Spring Authorization Server

If you need your own identity provider, **Spring Authorization Server** implements OAuth 2.1 and OpenID Connect 1.0 on top of Spring Security: authorization code + PKCE, client credentials, refresh tokens, device authorization, token exchange, introspection, revocation, JWKS and discovery endpoints, and dynamic client registration. Since **Spring Security 7.0 it ships as part of Spring Security** (module `spring-security-oauth2-authorization-server`). Boot 4 provides `spring-boot-starter-security-oauth2-authorization-server`, configured under `spring.security.oauth2.authorizationserver.*`. PKCE is required by default for authorization-code clients.

Choose it when you want the identity provider inside your Spring codebase, customised in Java. Choose Keycloak or a cloud IdP when you want an off-the-shelf product with admin UI, federation and user management included. Either way, *your APIs* look the same: resource servers configured with an `issuer-uri`. Running an authorization server needs a user store, persistent client and authorization storage (JDBC), and key management, so it is beyond this chapter's runnable examples.

11.15 Rate limiting and brute-force protection

Spring Security does **not** include rate limiting. Yet login and token endpoints are exactly what attackers hammer: password guessing, credential stuffing, and making you pay for slow bcrypt/Argon2 checks (10.5). Common layers:

Layer	Tool	Limits
edge	API gateway / WAF / CDN, Spring Cloud Gateway `RequestRateLimiter` (Redis token bucket)	requests per IP / client / API key
application	Bucket4j (token bucket, local or distributed via Redis/JCache) in a servlet filter	per user or per endpoint rules
login logic	count failures per account *and* per IP (listen to `AuthenticationFailureBadCredentialsEvent`), add delays, CAPTCHA or MFA after N failures	brute force on one account
identity provider	Keycloak/cloud IdPs have brute-force detection built in	when login happens at the IdP

Avoid hard **account lockout** after a few failures. It lets anyone lock out any user. Prefer progressive delays and step-up checks. Framework 7's `@ConcurrencyLimit` (Chapter 4) caps *concurrent* executions, not requests per second, so it is not a rate limiter. Algorithms (token bucket, sliding window) are explained in System Design Patterns — Rate Limiting.

11.16 Common mistakes (whole chapter)

Mistake	Consequence	Fix
no audience check	tokens for *other* APIs accepted	`jwt.audiences` property or `JwtClaimValidator("aud", ...)`
custom validator *replacing* defaults	expired tokens accepted	combine with `JwtValidators.createDefaultWithIssuer(...)`
`hasAuthority("messages.read")`	always 403	scopes are `SCOPE_messages.read`
`jwt()` tests without your converter	tests disagree with production	`.authorities(yourConverter)` or real tokens
long-lived JWT access tokens	stolen token valid for days	minutes + refresh tokens, or opaque tokens
tokens in `localStorage`	XSS = account takeover	BFF with `HttpOnly` cookie
accepting any `iss` in multi-tenant setups	attacker-run IdP accepted	allow-list of issuers
CSRF disabled on the BFF / OAuth2-login app	CSRF on a cookie session	keep CSRF on (`csrf.spa()`)
ID token sent to APIs	wrong audience, identity confusion	send the **access** token
rolling your own token server for real users	endless security work	use an authorization server
private key in `application.properties` / Git	anyone can mint tokens	vault/KMS, JWKS for public keys

11.17 In the interview

"How does a Spring resource server validate a JWT?" "`BearerTokenAuthenticationFilter` extracts the token from the `Authorization` header. `JwtAuthenticationProvider` calls the `JwtDecoder`, which Nimbus backs with the issuer's JWK Set found via `issuer-uri` discovery. It checks the algorithm, picks the key by `kid`, and verifies the signature. Then validators check `exp` and `nbf` with 60 seconds of skew, `iss`, `typ`, and the audience I configure. `JwtAuthenticationConverter` maps `scope` to `SCOPE_` authorities, plus any role claims I map. A failure returns 401 `invalid_token`, and a missing authority returns 403 `insufficient_scope`."

Follow-ups:

- *JWT or opaque token?* → JWT for local, fast validation inside the platform. Opaque when you need instant revocation or don't want claims exposed. The cost is an introspection call.
- *Why PKCE if the client has a secret?* → It binds the code to the party that started the flow, so a stolen code is useless. OAuth 2.1 requires it for all clients, and Spring Security 7 sends it by default.
- *How do you log a user out when tokens are stateless?* → Short access-token lifetimes, revoke the refresh token, end the local session, and use OIDC RP-initiated logout for the IdP session.
- *How do you store tokens in an SPA?* → Preferably not at all: a BFF keeps them server-side and gives the SPA an `HttpOnly` cookie with CSRF protection.
- *Service-to-service auth?* → Client credentials grant via `OAuth2AuthorizedClientManager` + `OAuth2ClientHttpRequestInterceptor`, or mTLS inside a mesh. Relay or exchange the user's token when

the user context matters.

- *What is `state` for?* → CSRF protection of the login callback. It ties the authorization response to the browser session that started the flow.

11.18 Security fundamentals beyond Spring

What it is

Everything in this chapter and the last one rests on a handful of cryptographic building blocks: hashes, symmetric and asymmetric keys, signatures, key exchange. Spring hides them behind `PasswordEncoder`, `JwtDecoder` and `https://` URLs, but interviews (and incident calls) expect you to know what they do. This section explains them from zero, shows each one with the plain JDK in `code/crypto-basics`, and then covers the protocols built from them: TLS, SSH keys, one-time codes, passkeys and single sign-on.

Analogy: a **hash** is a fingerprint (identifies, cannot be turned back into the person); **symmetric encryption** is a padlock where sender and receiver hold copies of the same key; **asymmetric encryption** is a mailbox with a public slot anyone can post into and a private key only the owner has; a **signature** is a wax seal only one ring can make but everyone can check.

Encoding vs hashing vs encryption vs tokenisation

	Purpose	Key?	Reversible?	Example	Wrong use
Encoding	change representation so bytes survive a channel	no	yes, by anyone	Base64, URL encoding, UTF-8	"hiding" a password in Base64
Hashing	fixed-size fingerprint; detect change; store passwords	no (HMAC adds one)	**no**	SHA-256, BLAKE3; for passwords bcrypt/Argon2 (`10.5`)	fast SHA-256 for passwords (GPU-crackable)
Encryption	confidentiality: only key holders can read	yes	yes, with the key	AES-GCM, ChaCha20-Poly1305, RSA-OAEP	encrypting passwords (then someone can decrypt them)
Tokenisation	replace sensitive data with a random stand-in; the real value lives in a secured vault	vault lookup	only via the vault	card number → `tok_8f3a…` in payment systems	—

An **HMAC** (hash-based message authentication code) is a hash mixed with a secret key: only key holders can compute or check it. HS256 JWTs and TOTP codes (below) are HMACs.

Symmetric vs asymmetric encryption

	Symmetric	Asymmetric (public-key)
Keys	one shared secret key	a key pair: **public** (share freely) + **private** (never leaves the owner)
Speed	very fast (AES has CPU instructions)	100–1000× slower; small data only
Problem	how do both sides get the key safely?	who does this public key really belong to?
Algorithms	AES-GCM, ChaCha20-Poly1305	RSA, elliptic curves (ECDH/X25519 for key exchange, ECDSA/Ed25519 for signatures)

Real systems are **hybrid**: asymmetric crypto agrees or protects a random symmetric key, and the bulk data goes through AES. **Authenticated encryption** modes (GCM, Poly1305) add a tag, so any tampering makes decryption fail rather than returning garbage — the demo proves it.

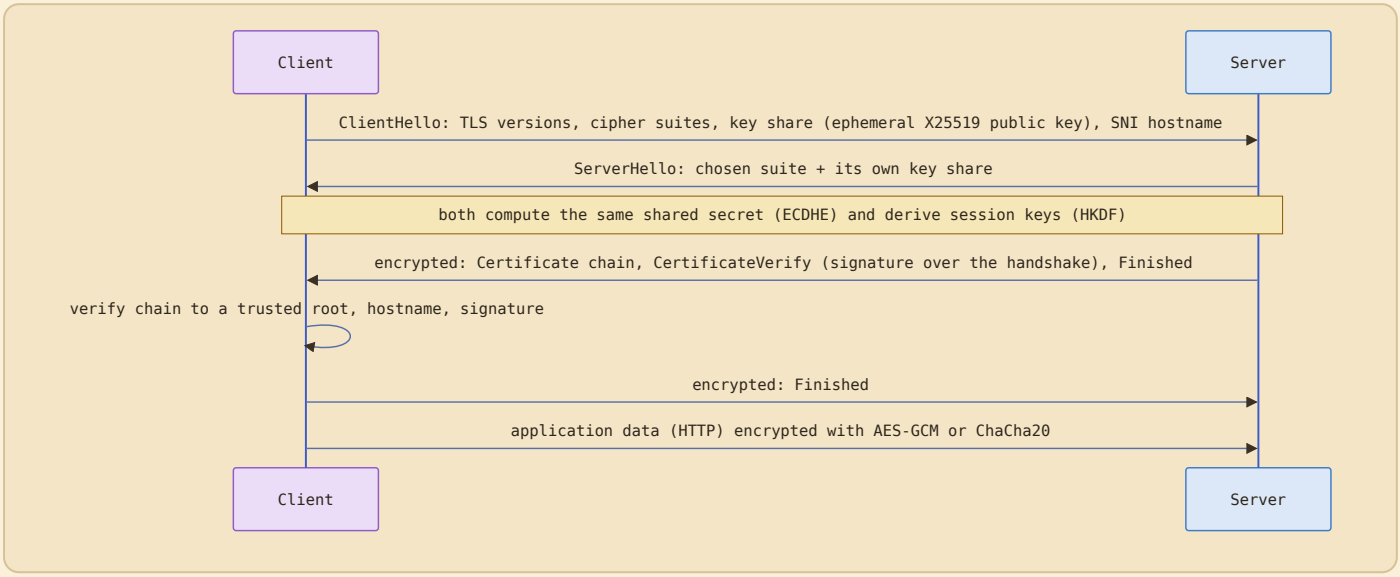
Digital signatures and certificates

A **digital signature** is computed with the private key over a hash of the message; anyone with the public key can verify that (1) the holder of that private key signed it and (2) not one bit changed. That is exactly how an RS256/ES256 JWT is trusted (§11.1).

A **certificate** (X.509) is a signed statement "this public key belongs to `api.example.com`", signed by a **certificate authority (CA)**. Browsers and the JDK ship a list of trusted root CAs; a server sends its certificate plus intermediates, and the client checks the **chain** up to a trusted root, the name, and the validity dates.

The TLS 1.3 handshake

HTML book §1.4 says what TLS gives (confidentiality, integrity, server authentication). Here is how, in one round trip (RFC 8446):



Key ideas: the **key exchange** (ECDHE) gives both sides a secret nobody on the wire can compute; the **signature** in `CertificateVerify` proves the server owns the certificate's private key; the keys are **ephemeral**, so recording traffic today and stealing the server key later reveals nothing (**forward secrecy**). **mTLS** (mutual TLS) adds a client certificate so the server authenticates the client too — common between services (§11.10). In Spring Boot, TLS is configured with SSL bundles (`server.ssl.bundle`), usually terminated at a load balancer or ingress instead.

SSH keys

SSH logs you into servers and Git hosts. `ssh-keygen -t ed25519` creates a key pair; you put the **public** key in the server's `~/.ssh/authorized_keys` (or your GitHub settings). On login the server sends a challenge, your client **signs** it with the private key, and the server verifies with the stored public key — no password crosses the wire. The first time you connect, SSH shows the **server's** host-key fingerprint; accepting it stores it in `known_hosts`, and a later mismatch warns of a possible man-in-the-middle. Protect the private key with a passphrase and `ssh-agent`; never copy it between machines.

TOTP: the six digits in an authenticator app

TOTP (time-based one-time password, RFC 6238) builds on **HOTP** (RFC 4226):

1. At enrolment the server generates a random secret and shows it as a QR code (`otpauth://totp/...?secret=BASE32...`); the app stores it. Both sides now share a secret.
2. Every 30 seconds both compute `counter = floor(unixTime / 30)`, then `HMAC-SHA1(secret, counter)`.
3. **Dynamic truncation**: the last 4 bits of the HMAC pick an offset; 31 bits from there, `mod 10^6`, give six digits.
4. The server accepts the current step and usually ±1 step for clock drift, and should remember used codes so one cannot be replayed.

`Totp.java` implements it in 30 lines; `TotpTest` checks every RFC 6238 Appendix B test vector (SHA-1, SHA-256 and SHA-512) and the RFC 4226 HOTP vectors.

```
byte[] h = mac.doFinal(ByteBuffer.allocate(8).putLong(counter).array()); // HMAC(secret, 8-byte counter)
int offset = h[h.length - 1] & 0x0f;                                     // dynamic truncation
int binary = ((h[offset] & 0x7f) << 24) | ((h[offset + 1] & 0xff) << 16)
           | ((h[offset + 2] & 0xff) << 8) | (h[offset + 3] & 0xff);
return String.format("%0" + digits + "d", binary % (int) Math.pow(10, digits));
```

TOTP is phishable (a fake site can ask for the code and replay it within 30 s) and depends on a shared secret stored on the server. That is what passkeys fix.

Passkeys (WebAuthn / FIDO2)

A **passkey** is a key pair created by your device (phone, laptop, security key) **for one website**. Registration: the browser's WebAuthn API asks the device's **authenticator** to create a key pair scoped to the site's domain (the **relying party ID**); the site stores only the public key. Login: the site sends a random challenge; the authenticator checks you (fingerprint, face, PIN), **signs** the challenge together with the origin, and the site verifies with the stored public key.

Why it is stronger than passwords + TOTP: nothing secret is stored on the server (a database leak gives attackers only public keys), and the signature covers the origin, so a look-alike phishing domain gets a signature that is useless on the real site. Synced passkeys (iCloud Keychain, Google Password Manager) survive losing a phone. Spring Security supports them since 6.4 with `http.webAuthn(w -> w.rpName("Shop").rpId("shop.example.com").allowedOrigins("https://shop.example.com"))` (module `spring-security-webauthn` in 7.x), and records a `FACTOR_WEBAUTHN` authority for multi-factor rules (10.4).

Single sign-on: SAML vs OpenID Connect

SSO means logging in once at an identity provider (IdP) and being let into many applications (service providers / relying parties) without new passwords.

	SAML 2.0 (2005)	OpenID Connect (2014, on OAuth 2.0)
Token	signed **XML assertion**	signed **JWT** ID token (§11.2)
Transport	browser POSTs/redirects the assertion to the app	authorization-code flow + PKCE, back-channel token call
Fits	enterprise web apps, older corporate IdPs (AD FS)	web, SPA/BFF, mobile, APIs; also gives access tokens
Spring	`saml2Login()` (`spring-security-saml2-service-provider`)	`oauth2Login()` (§11.9)

Both rely on the same trick: the IdP **signs** a statement about the user, and the app verifies the signature with the IdP's published key or certificate. For new systems pick OIDC; add SAML when an enterprise customer's IdP only speaks SAML (Keycloak, Okta and Entra ID speak both and can bridge).

Seeing it: `code/crypto-basics`

Plain JDK 25, no Spring beans. `mvn -q -B -ntp verify` runs 13 JUnit tests (RFC vectors, AES-GCM tamper detection, signatures, X25519 agreement); then `java -cp target/classes handbook.security.CryptoBasicsDemo` prints:

```
encode  Base64      : cGF5IGJvYiAxMDA=  -> decodes back to: pay bob 100
hash    SHA-256     : aa843d8373c66e26...  ('pay bob 900' -> 70ef8a2487a6d483...)
encrypt AES-256-GCM : 27 bytes (11 plaintext + 16 auth tag) -> decrypts to: pay bob 100
encrypt tampered    : AEADBadTagException (integrity check failed)
sign    Ed25519     : 64-byte signature; verify original = true, verify 'pay bob 900' = false
ECDHE   X25519      : both sides derived the same 32-byte secret = true (e4582f7aff4d...)
ECDHE   record      : server decrypts client's record: GET /
TOTP    t=59          : counter=1         code=94287082
TOTP    t=1111111109  : counter=37037036  code=07081804
TOTP    t=1234567890  : counter=41152263  code=89005924
TOTP    t=2000000000  : counter=66666666  code=69279037
TOTP    6-digit app : 921300 valid 25 s later = true, valid 5 min later = false
```

The four `TOTP t=` codes are exactly the SHA-1 column of RFC 6238 Appendix B. Keys and the X25519 secret are random on every run.

Common mistakes

- Calling Base64 "encryption" → it is encoding; anyone can decode it.
- Hashing passwords with plain SHA-256 or MD5 → use a slow, salted password hash (bcrypt, Argon2, PBKDF2).
- Reusing an AES-GCM IV (nonce) with the same key → breaks confidentiality and integrity; use a random 12-byte IV per message.
- Rolling your own crypto or protocol → use the JDK, TLS, and vetted libraries; your job is choosing parameters and managing keys.
- Disabling certificate checks ("trust all") to make a test pass → it removes the *authentication* half of TLS in production too.
- Comparing codes or MACs with `equals` → use a constant-time comparison (`MessageDigest.isEqual`) to avoid timing leaks.

In the interview

30-second answer — how HTTPS works: "TLS 1.3 does an ephemeral Diffie-Hellman key exchange in the first round trip, so client and server derive the same session keys without sending them. The server proves its identity by signing the handshake with the private key matching its

certificate, and the client checks the certificate chain up to a trusted CA and the hostname. After that, data is encrypted and integrity-protected with a fast symmetric cipher such as AES-GCM. Ephemeral keys give forward secrecy."

- *Hashing vs encryption?* → Hashing is one-way and keyless (fingerprints, password storage); encryption is reversible with a key (confidentiality).
- *Why both symmetric and asymmetric?* → Asymmetric solves key distribution and identity but is slow; symmetric is fast. TLS uses the first to set up the second.
- *How does an authenticator app know the right code offline?* → Shared secret + current time → HMAC → truncation (TOTP, RFC 6238).
- *Why are passkeys phishing-resistant?* → The authenticator signs a challenge bound to the site's origin, and the server holds only a public key.
- *SAML vs OIDC?* → Both are SSO with an IdP-signed statement; SAML uses XML assertions via browser POST, OIDC uses JWT ID tokens over OAuth 2.0 and suits mobile and APIs.

Self-check

1. Which primitive protects a JWT's claims from being changed, and which would hide them? → A signature (JWS) / encryption (JWE).
2. What gives TLS forward secrecy? → Ephemeral (EC)DHE key exchange; the long-term key only signs.
3. What does the server store for a passkey? → The user's public key and a credential ID.
4. Why does a TOTP server accept ±1 step? → To tolerate clock drift between phone and server.

TL;DR

Encoding is reversible without a key, hashing is one-way, encryption needs a key, tokenisation swaps data for a vault reference. Symmetric = one fast shared key; asymmetric = public/private pair for key exchange and signatures; real systems combine them (TLS 1.3: ECDHE + certificate signature + AES-GCM). SSH and passkeys log in by signing a challenge; TOTP = HMAC(secret, time/30) truncated to six digits. SSO: SAML (XML) or OIDC (JWT) — prefer OIDC for new work.

Sources: RFC 6238 (TOTP), RFC 4226 (HOTP), RFC 8446 (TLS 1.3), RFC 4251/4252 (SSH), RFC 7748 (X25519), RFC 8032 (Ed25519), NIST SP 800-38D (GCM), W3C Web Authentication Level 2, OASIS SAML 2.0 Core, OpenID Connect Core 1.0, Spring Security reference (Passkeys, SAML 2.0 Login), JDK 25 `java.security` / `javax.crypto` API docs.

Chapter 11 TL;DR

- **Tokens**: a JWT is `header.payload.signature`. Claims are readable, and the signature is what makes them trustworthy. Prefer asymmetric keys (RS256/ES256) published as a JWKS.
- **OAuth 2.0/OIDC**: authorization code + PKCE for users, client credentials for services, refresh tokens for renewal. Implicit and password grants are gone (Spring Security 7 removed password-grant support).
- **Resource server**: `oauth2ResourceServer(jwt)` + `issuer-uri`. It validates signature, `exp`/`nbf` (60 s skew), `iss`, `typ`, and your `aud`. 401 `invalid_token` vs 403 `insufficient_scope`. It publishes RFC 9728 metadata.
- **Authorities**: `scope` → `SCOPE_x` by default. Map role claims with a converter or Boot properties.
- **Testing**: real tokens from a local `NimbusJwtEncoder` for decoder rules, `jwt()`/`opaqueToken()`/`oidcLogin()` for access rules. Pass your converter to `jwt()`.
- **Variants**: opaque tokens via introspection (instant revocation). Multi-tenant via `JwtIssuerAuthenticationManagerResolver` with an issuer allow-list.

- **Clients**: `oauth2Login` (state, nonce, PKCE, tokens server-side), `OAuth2AuthorizedClientManager` + `RestClient` interceptor or `@ClientRegistrationId` HTTP interfaces, token relay or exchange.
- **Browsers**: BFF + `HttpOnly` cookie + CSRF, and no tokens in `localStorage`.
- **Operations**: keys in a vault and rotated via `kid`, Spring Authorization Server (now inside Spring Security 7) if you must run an IdP, rate limiting at the edge.
- **Fundamentals** (§11.18): encoding ≠ hashing ≠ encryption; hybrid crypto (asymmetric key exchange + signatures, symmetric AES-GCM for data) is how TLS 1.3 works; SSH keys and passkeys sign a challenge; TOTP = HMAC(secret, time/30) truncated; SSO via SAML (XML) or OIDC (JWT).

Next: Chapter 12 — Testing Spring apps covers the test slices used here in depth.